CLAVIX: Deterministic, Closed-Loop Voice-to-Action Automation of the Windows Desktop

Krish Kamra Shreyash Pathak Varun Agrawal Aditya Angane
Department of Artificial Intelligence, Mukesh Patel School of Technology Management & Engineering
NMIMS University, Mumbai, Maharashtra, India
{krish.kamra78,shreyash.pathak53,varun.agrawal60,aditya.angane33}@nmims.in

***Abstract*—Voice assistants can answer questions but rarely operate the desktop applications people actually use. We present CLAVIX, a voice-to-action agent for Windows 11 that turns one spoken command into a validated plan of typed actions and executes it in a closed loop: every step is dispatched to the most deterministic route available (native operations, then accessibility-tree grounding, then coordinate fallback) and then checked against an explicit postcondition on operating-system state, with bounded retries. All models run locally. On a ten-task benchmark spanning deterministic, sequential and UI-grounded tasks, CLAVIX completes all 50 runs with reference plans and all 30 with plans from a local `llama3:8b`, against 7 of 10 without accessibility grounding. From recorded speech, every command is transcribed exactly and 44 of 50 runs succeed. On-device recognition (Faster-Whisper `base.en`, CPU) needs a median of 441 ms after end of speech in isolation and 395 ms in live use, while LLM planning takes 90 % of the end-to-end time. We report the engineering lessons behind these numbers: why accessibility search must be bounded per window, why exact label matches must outrank partial ones, and how plan validation and an ablated prompt removed planner errors without tuning to the test set. With `mistral:7b` the same prompt reaches 21 of 30. The code, benchmark suite and recorded commands are open source at https://github.com/KrishKamra/Clavix.**

***Index Terms*—voice user interfaces, desktop automation, large language models, UI accessibility, speech recognition**

I. INTRODUCTION

Voice assistants answer questions well, but they rarely operate the desktop applications where people do their work. Doing so hands-free matters for users with motor impairments, and it is convenient for everyone whose hands are busy. Large language models can now turn a spoken request into a sequence of actions [1], and agents that operate graphical interfaces are an active research area [2], [3]. Two properties are still hard to get. The first is *reliability*: an agent that acts open-loop cannot tell that a click missed or a file was never written, so it fails silently. The second is *locality*: sending speech and screen contents to a cloud service is often unacceptable for private desktops.

CLAVIX addresses both. It runs speech recognition and planning on the device, prefers deterministic operations (shell commands, URLs) over interface manipulation, finds interface elements through the operating system's accessibility tree instead of pixels, and checks every step against an explicit postcondition on operating-system state before moving on.

Our contributions are:

- a closed-loop agent architecture that couples a schema-constrained LLM planner with a tiered, deterministic-first dispatcher and an OS-state verifier (Section III);
- an on-device acoustic front end with energy endpointing whose ASR latency excludes the end-of-speech hang by construction (Section III-B);
- a benchmark of ten Windows tasks in three tiers with machine-checkable ground truth, run with reference plans and with LLM plans, from text and from recorded speech (Section IV);
- an open-source implementation with conformance-tested module contracts and continuous integration on Windows and Linux.[1]

II. RELATED WORK

LLM agents. ReAct [1] interleaves reasoning with tool calls. CLAVIX instead asks the model for a complete, schema-validated plan over a closed set of typed actions and keeps execution and checking outside the model, so that every action is inspectable before it runs.

Desktop and GUI agents. UFO [2] operates Windows applications through their user interfaces, and OSWorld [3] benchmarks multimodal agents in real computer environments. CLAVIX is narrower: it targets spoken commands, runs entirely on local models, uses the accessibility tree (UI Automation [4]) rather than screenshots to find elements, and verifies each step against operating-system state.

Speech recognition and local models. Whisper [5] provides robust recognition; we use the CTranslate2 reimplementation Faster-Whisper [6] for low-latency CPU inference. Open-weight models such as Llama 3 [7] and Mistral 7B [8], served locally by Ollama [9], make on-device planning practical.

III. SYSTEM DESIGN

Fig. 1 shows the pipeline. The core depends only on three small module interfaces (acoustic, grounding, feedback), each with a conformance test suite, and the concrete implementation of each is chosen in configuration.

A. Agent state machine

The agent is an explicit finite-state machine (Fig. 2). Illegal transitions raise, and failures are recorded in the task outcome rather than thrown, so every run, including aborted ones, produces a complete telemetry record of state transitions and per-step timings.

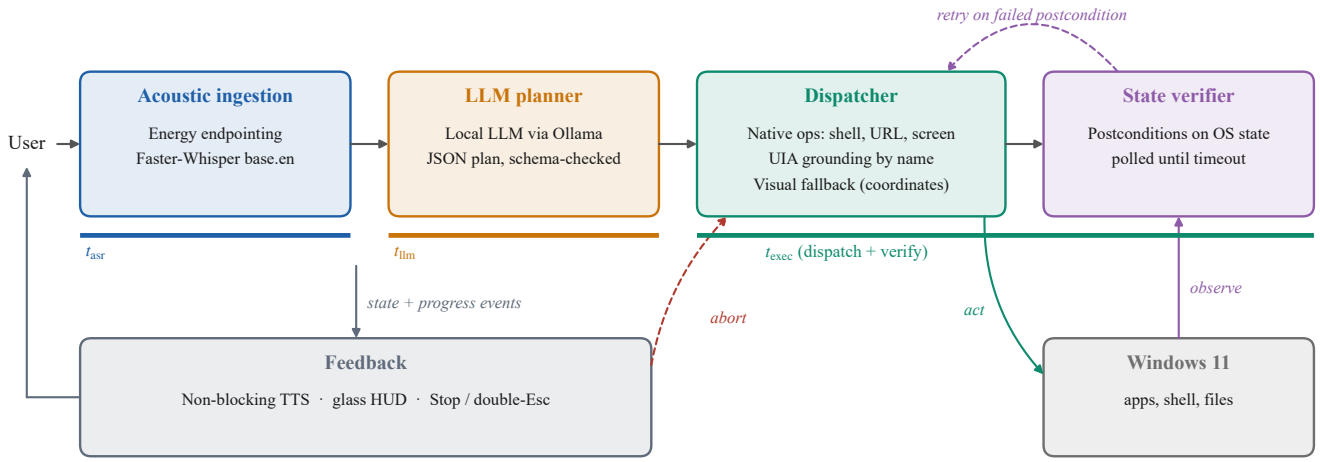


Fig. 1. Clavix architecture. A spoken command is endpointed and transcribed on device (t_{asr}), planned by a local LLM into a schema-checked JSON plan (t_{llm}), then each step is dispatched and its postcondition verified against OS state, with retries (t_{exec}). Feedback runs alongside: spoken summaries and a glass HUD mirror the agent's state, and a Stop button or double-Esc aborts the running step.

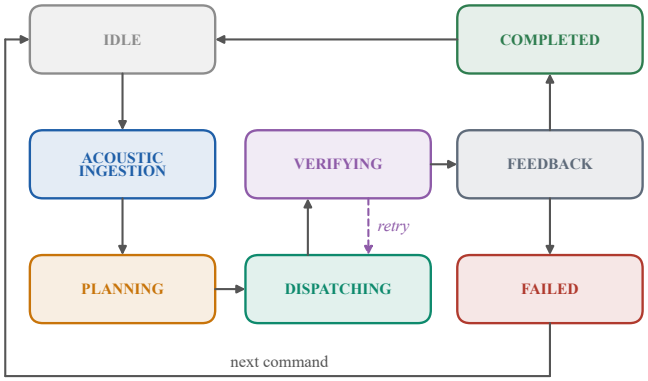


Fig. 2. Agent states. A step whose postcondition fails is re-dispatched up to a retry budget. An error or abort in any working state (ingestion to verification) moves straight to FEEDBACK and ends the task as FAILED; every transition is reported to the feedback module.

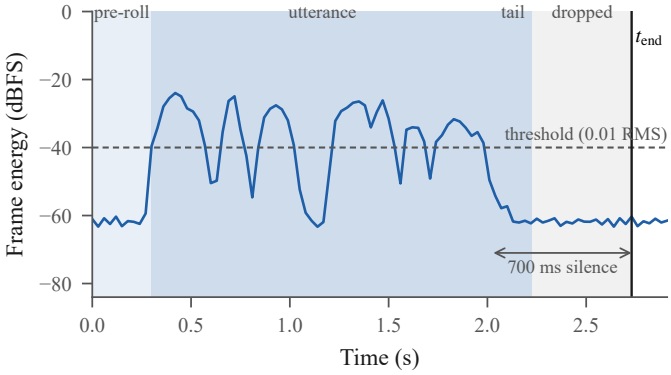


Fig. 3. Energy endpointing on a recorded command ("run echo clavix ready"), padded with 1 s of the recording's own room noise. Speech starts after 90 ms above threshold; 300 ms of pre-roll is kept. The utterance ends after 700 ms below threshold (t_{end}), and only 200 ms of that silence is passed to the recognizer.

B. Acoustic ingestion

Audio is captured at 16 kHz in 30 ms frames. A frame counts as speech when its RMS energy reaches a threshold of $\max(0.01, 3\bar{e})$, where $\bar{e}$ is the ambient RMS measured once at start-up (Fig. 3). The utterance is transcribed by Faster-Whisper [6] (`base.en`, 8-bit integer weights on CPU, greedy decoding, Silero VAD filtering) on a dedicated worker thread. We define t_{asr} as the time from end-of-speech detection to the final transcript, so the 700 ms hang is a configuration constant reported separately, not hidden inside t_{asr}.

C. Planning

The planner prompts a local LLM served by Ollama [9] to emit a JSON plan over a closed vocabulary of five actions: `launch_url`, `run_command`, `take_screenshot`, `click_element` and `speak_feedback`. Each step names its route (deterministic, OS grounding, visual fallback or feedback), a typed payload, and an optional postcondition. Plans are validated against a strict schema; a validation error is returned to the model for up to two repair rounds.

D. Dispatch and verification

The dispatcher executes each step through the most deterministic route available. Shell commands pass a configurable safety policy that blocks destructive patterns. Clicks first resolve the target by accessible name in the Windows UI Automation tree [4] and fall back to screen coordinates only when that fails and the fallback is enabled. After dispatch, the verifier polls one of five postconditions on OS state (`window_title_contains`, `process_running`, `file_exists`,

[1]https://github.com/KrishKamra/Clavix

exit_code, stdout_contains) until it holds or a timeout expires.

E. Structural grounding

A click_element step names its target by label ("File", "Start"). Grounding normalises the label (case, access-key markers, shortcut suffixes, a trailing ellipsis) and scores it against each element's accessible name and automation ID: 1.0 for an exact match, 0.9 when the name starts with the label, 0.85 when it contains it as a whole word, otherwise a string-similarity ratio, accepting scores of at least 0.8. Ties go to interactive control types, then smaller and higher elements. The search visits the foreground window, then the taskbar, then the other visible windows front to back, but an exact match anywhere beats a partial match in an earlier scope. Without that rule, with Notepad behind another window, "File" matched the taskbar's "File Explorer – 1 running window" (a prefix match) and opened File Explorer. The element is activated through UI Automation patterns (invoke, expand/collapse, toggle, select), with a coordinate click as the last resort.

Real desktops forced three robustness measures. Accessibility providers vary by orders of magnitude: the desktop window took about 10 s to walk for 88 elements, and a browser showing a documentation page exposed 8,311 elements and took 3.6 s. Searches therefore check their deadline between individual UIA calls, search the desktop window last, and give each window at most 40 % of the time budget per pass, so one large window cannot starve the rest. Top-level windows are listed through Win32 EnumWindows (visible, uncloaked) rather than the UIA raw view, which also walks hidden windows and blocked indefinitely on a busy desktop. With an ordinary window in front, "Start" resolves in 118 ms to 139 ms; behind that browser, in 1.3 s.

F. Feedback and safety

Spoken feedback runs on its own thread so that it never stalls the agent. A glass head-up display (Fig. 4) mirrors the agent: the transcript, the plan's steps as they run, and the latency of each stage. The agent publishes these as structured progress events, and the HUD renders them in a separate process, so a crash or stall in the interface cannot affect the agent. An automation overlay has duties an ordinary window does not: it never takes keyboard focus (or keystrokes and clicks would land in the wrong application), it is excluded from screen capture (or it would pollute screenshots taken by the agent and the coordinate fallback), and the grounding search skips it (its own Stop button must never be matched). A Stop button or a double press of Escape aborts the running step. The measured prototype starts in 1.2 s and uses 157 MB of private memory and 0.16 % CPU when idle. The final module speaks through a bounded queue on a dedicated thread, so speak returns at once; with speech, HUD and hotkey live, the reference benchmark still completes 10 of 10 tasks, and grounding never matches the HUD.

IV. Evaluation

A. Benchmark and setup

Table I lists the ten tasks. Each is run with the *reference* planner, which replays a hand-written plan and so isolates execution, and with LLM planners. Commands are given either as text or as recordings of the spoken command (Table II). Each task is repeated (five times with the reference planner, three with the LLM), each repetition in a fresh workspace directory that is also the working directory, so relative paths in an LLM plan land where ground truth looks; processes a task started (Notepad) are terminated afterwards. Ground truth is checked independently of the agent's own verifier, and every failure was inspected rather than only counted: one harness bug that scored four correct tasks as failures was found this way.

B. Speech recognition

Table III and Fig. 5 compare two model sizes. tiny.en is twice as fast but mishears domain words ("github" as "get up", "file" as "sign"), which change the plan; base.en transcribes all fifty recordings exactly, so we use it by default.

C. Spoken commands end to end

Played from the recordings (5 runs per task), all 50 commands were transcribed exactly, and 44 of 50 runs succeeded: Tier 1 15/20, Tier 2 14/15, Tier 3 15/15. Five failures are T1-02, in every run, where a perfect transcript still changed the action (Section V); one was an LLM request that exceeded its 60 s timeout. Median end-to-end time was 10.7 s, of which planning took 90 % (Fig. 6); execution adds 0.1 s to 0.2 s for Tiers 1–2 and 0.7 s for Tier 3, where grounding searches the accessibility tree. ASR took a median of 0.76 s in this run against 441 ms in isolation (Table III): it ran after hours of CPU-bound LLM inference, the thermal effect discussed in Section V. The latency budget is therefore dominated by the local LLM, not by speech or execution.

D. Task success

With reference plans and UIA grounding, all 50 runs succeed (Table IV), against 7/10 in the Phase 1 baseline without grounding; median execution time is 152 ms for Tier 1, 173 ms for Tier 2 and 886 ms for Tier 3 (p95 3.7 s, when a large browser window is in front). With llama3:8b planning from the command text, all 30 runs succeed after the prompt changes of Table V, each measured on its own. The first two few-shot examples fixed the errors they targeted (a dropped confirmation, a detached mkdir) but caused a new one (cd workspace), fixed by stating that the workspace is the current directory. Removing two examples that repeated benchmark commands verbatim exposed a planning error the leak had hidden; treating a coordinate click without coordinates as an invalid plan let the existing repair round fix it. Plans match the reference action sequence in every run, while full payload matches stay at 40 % because equivalent commands differ in wording (echo hello > note.txt against an absolute path). With the same prompt, mistral:7b succeeds in 21 of 30 runs (action

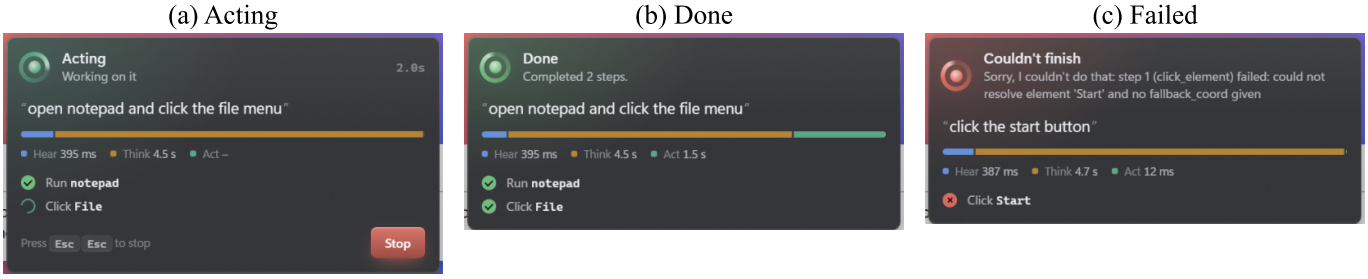


Fig. 4. The glass HUD (prototype on Windows 11, real Acrylic backdrop). Its colour is the pipeline stage, and the bar under the transcript is the latency breakdown of Fig. 1 filling in live (**Hear**, **Think**, **Act**). (a) Acting, with the plan's steps and the Stop control; (b) done; (c) a failed step and the reason.

TABLE I
BENCHMARK SUITE. TIER 1 TASKS NEED ONE DETERMINISTIC ACTION, TIER 2 CHAIN SEVERAL, AND TIER 3 NEED UI GROUNDING. GROUND TRUTH IS CHECKED ON OS STATE AFTER THE RUN.

Task	Tier	Spoken command	Reference actions
T1-01	1	open github	`launch_url`
T1-02	1	run echo clavix ready	`run_command`
T1-03	1	create a file called note.txt in the workspace containing hello	`run_command`
T1-04	1	take a screenshot and save it to the workspace as screen.png	`take_screenshot`
T2-01	2	make a folder called reports in the workspace and put a file summary.txt in it saying done	`run_command` ×2
T2-02	2	open the python docs and tell me when it's open	`launch_url`, `speak_feedback`
T2-03	2	save the text checkpoint to log.txt in the workspace, then screenshot the top left corner to corner.png	`run_command`, `take_screenshot`
T3-01	3	open notepad and click the file menu	`run_command`, `click_element`
T3-02	3	click the start button	`click_element`
T3-03	3	take a screenshot to before.png in the workspace, then click the search box	`take_screenshot`, `click_element`

TABLE II
EXPERIMENTAL SETUP.

OS	Windows 11 (26H2)
CPU	Intel Core Ultra 9 285H (16 cores, hybrid)
ASR	Faster-Whisper 1.2.1, `base.en`, int8, beam 1
LLM planners	`llama3:8b`, `mistral:7b` (Ollama 0.40.13, temp. 0)
Grounding	Windows UI Automation (uiautomation 2.0.29)
Microphone	Bluetooth headset (WH-CH520)
Speech	10 commands, one speaker, quiet room

TABLE III
ASR ON THE TEN RECORDED COMMANDS (5 PASSES, $n = 50$).

Model	Median t_{asr} (ms)	p95 t_{asr} (ms)	WER (%)	Exact
`tiny.en`	210	239	17.5	25/50
`base.en`	441	495	0.0	50/50

WER over case-folded words without punctuation. CPU, int8, greedy decoding, model warmed up.

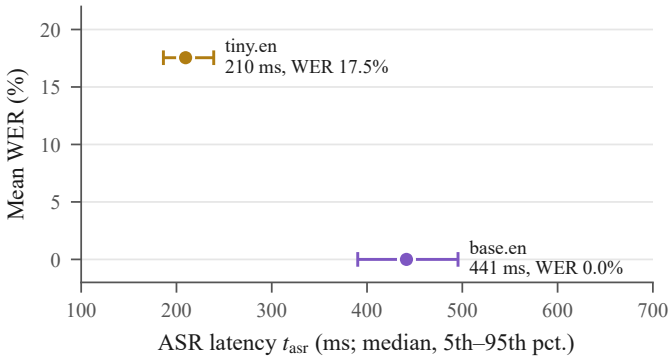


Fig. 5. Accuracy against latency on the recorded commands. Whiskers span the 5th to 95th percentile of t_{asr}. `base.en` doubles the latency of `tiny.en` but removes every word error.

match 72 %). Its errors are planning errors, mostly in Tier 3: asked to open Notepad and click its File menu, it opened the website notepad.com, and it named the taskbar's "Search" button "Search box", below the matching threshold. The prompt was developed with `llama3:8b`, so part of the gap may be prompt fit rather than model quality.

E. Grounding ablation

Accessibility grounding is what makes Tier 3 possible. With the same reference plans and grounding disabled (the Phase 1 configuration, where only coordinate clicks remain), no Tier 3 task succeeds (0/3); with UI Automation grounding, all 15 runs succeed and every click lands on the intended element ("File", "Start", "Search"), with no coordinate fallback used. A coordinate-only agent would need screen positions that neither

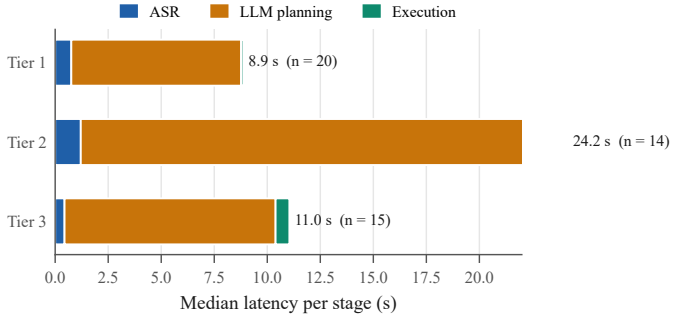


Fig. 6. Median latency per stage from spoken command to verified action (recorded commands, `llama3:8b` on CPU, UIA grounding; one run lost to an LLM timeout is excluded). Planning is 90 % of the median run.

TABLE IV
TASK SUCCESS PER TIER (SUCCESSFUL RUNS / RUNS).

Configuration	Tier 1	Tier 2	Tier 3	All
Reference plans, no grounding[a]	4/4	3/3	0/3	7/10
Reference plans, UIA grounding	20/20	15/15	15/15	50/50
`llama3:8b` plans, UIA grounding	12/12	9/9	9/9	30/30
same, spoken commands	15/20	14/15	15/15	44/50
`mistral:7b` plans, UIA grounding	12/12	6/9	3/9	21/30

[a] Phase 1 baseline (stub grounding, one run). Reference plans and spoken commands: 5 runs per task; LLM from text: 3 runs. Spoken commands are played from the recordings (Section IV-C).

TABLE V
PLANNER PROMPT ABLATION (`llama3:8b`, 3 RUNS PER TASK). EACH ROW ADDS ONE CHANGE.

Prompt	Tiers 1–2	Tier 3[a]	Plan match[b]
Phase 1 prompt	18/21	–	90 / 40
+ two few-shot fixes	18/21	–	90 / 40
+ "workspace is the current directory"	21/21	–	100 / 40
+ no benchmark commands in examples	21/21	6/9	90 / 30
+ no guessed window titles	21/21	9/9	100 / 40

[a] Measured once grounding was integrated. Row 4: before plan validation (T3-02 planned as a coordinate click without coordinates). Row 5 also treats such clicks as invalid plans.
[b] Percent of plans whose action sequence / full payload matches the reference.

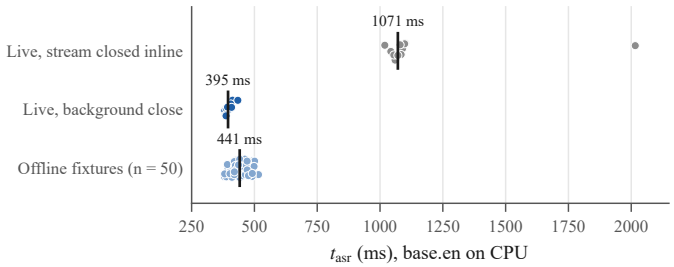


Fig. 7. Live ASR latency before and after moving the microphone stream's close off the critical path, compared with offline recordings. Closing a Bluetooth input stream took about 570 ms and had been counted in t_{asr}. The 2 016 ms outlier was a clipped capture.

the user nor the planner knows: T3-01 and T3-02 carry none, and T3-03's fixed fallback coordinate is correct only for one screen layout.

V. DISCUSSION AND LIMITATIONS

Measure the live path, not only fixtures. Offline, t_{asr} was 441 ms, but the first live run measured a median of 1 071 ms. The offset was constant across utterance lengths, which ruled out the model: closing the Bluetooth input stream blocked transcription. Closing it on a background thread brought live t_{asr} to 395 ms (Fig. 7).

Synthetic speech is not a proxy for users. We first benchmarked commands rendered with a desktop TTS voice. Its error went in opposite directions for the two models: `tiny.en` scored 6.8 % WER on synthetic speech but 17.5 % on human recordings, while `base.en` scored 4.7 % and 0.0 % (the voice said "github" so that it was heard as "guess up"). All reported ASR results therefore use human recordings.

Variance on hybrid CPUs. On a laptop with performance and efficiency cores, median t_{asr} varied between runs (386 ms to 441 ms for `base.en`); we therefore compare configurations within one run.

A perfect transcript can still change the action. With recorded speech, every command was transcribed exactly (WER 0 after case and punctuation normalisation), yet T1-02 failed where it passes from text: Whisper's capitals and full stop ("Run Echo Clavix Ready.") reached the `echo` command, and the output no longer matched. WER alone understates the effect of ASR on actions, so we report task success from audio as well.

Few-shot examples have side effects. Two examples fixed the planner errors we targeted (a dropped confirmation and a detached `mkdir`), raising Tier 2 from 6/9 to 9/9, but the `mkdir` example led the model to append `cd workspace` to another task. A one-line rule restored it. We measured each prompt change separately (Table V).

Threats to validity. Two of the original few-shot examples were benchmark commands verbatim. We replaced them and report only the decontaminated prompt; a test now rejects any example that repeats a benchmark command. Without the leaked example, the planner chose coordinate fallback without coordinates for "click the start button", a weakness the leak had hidden; treating such plans as invalid let the repair round fix it (Table V). Latency also depends on machine state: after about 40 minutes of CPU-bound LLM inference, median t_{asr} rose to 709 ms, so we compare configurations within one run and report the thermal effect where it applies.

Labels need application context. Grounding resolves a target by its accessible name, so the same label in two applications is ambiguous: with Notepad and a PDF viewer both showing an exact "File", resolving before Notepad's window existed matched the viewer's button. The agent avoids this only because the plan first waits for Notepad's window (`window_title_contains:Notepad`), after which front-to-back search finds Notepad's menu. A window hint in `click_element` ("File in Notepad") would remove the ambiguity and is future work.

Guessed postconditions. The LLM sometimes added a window-title check it could not know (for example "Windows

Start menu" after clicking Start), so a successful click failed verification. A prompt rule against guessed titles raised LLM-planned success from 27/30 to 30/30 while keeping every file, output and launched-application check.

Limitations. The benchmark has ten tasks, and its recordings come from one speaker with one microphone in a quiet room, in English. Clavix runs on Windows only. The action vocabulary has no text entry yet, and planning with an 8-billion-parameter model on a laptop CPU takes several seconds per command. Spoken feedback uses the system's SAPI voices, which sound less natural than neural speech.

VI. Conclusion

Clavix shows that a fully local, voice-driven agent can operate the Windows desktop reliably when planning is constrained by a schema, execution prefers deterministic routes, interface elements are found through the accessibility tree, and every step is verified against operating-system state. With these pieces it completes every benchmark run from text commands, and 44 of 50 from recorded speech. The work also shows where the remaining cost and fragility lie: local LLM planning dominates latency, accessibility search must be bounded against slow providers and ambiguous labels, and a perfect transcript can still alter an action. Future work includes text entry and keyboard actions, a window hint for element targets, a larger benchmark with more speakers, and faster planning models. All code, the benchmark and its recordings are available at https://github.com/KrishKamra/Clavix.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[2] C. Zhang *et al.*, "UFO: A UI-focused agent for Windows OS interaction," arXiv:2402.07939, 2024.

[3] T. Xie *et al.*, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[4] Microsoft, "UI Automation overview," https://learn.microsoft.com/en-us/windows/win32/winauto/entry-uiauto-win32, 2025.

[5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 28 492–28 518.

[6] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," https://github.com/SYSTRAN/faster-whisper, 2025, version 1.2.1.

[7] A. Dubey *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[8] A. Q. Jiang *et al.*, "Mistral 7B," arXiv:2310.06825, 2023.

[9] Ollama, "Ollama: Run large language models locally," https://ollama.com, 2025.